

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25 %)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15 %)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation (10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

CO4_AT2

**AI-Based Smart Renewable Energy Asset Performance
Monitoring System**

NAME: **RAKKESH KARAN R**

REG NO: **192524253**

DEPT: **BACHELOR OF TECHNOLOGY IN AIDS**

COURSE: **CSA1017 SOFTWARE ENGINEERING**

CI/CD Pipeline Design for AI-Based Smart Renewable Energy Asset Performance Monitoring System

1. Introduction

The **AI-Based Smart Renewable Energy Asset Performance Monitoring System** is designed to continuously monitor solar panels, wind turbines, and battery storage systems using IoT sensors, AI/ML models, cloud computing, and data analytics.

A CI/CD pipeline is required to automate software development activities such as code integration, compilation, testing, security checking, packaging, and deployment. The pipeline ensures that new features and corrections can be delivered quickly while maintaining reliability, security, and performance.

The proposed CI/CD pipeline uses **GitHub, GitHub Actions, Docker, SonarQube, PyTest/JUnit, OWASP tools, and Kubernetes/cloud deployment** technologies.

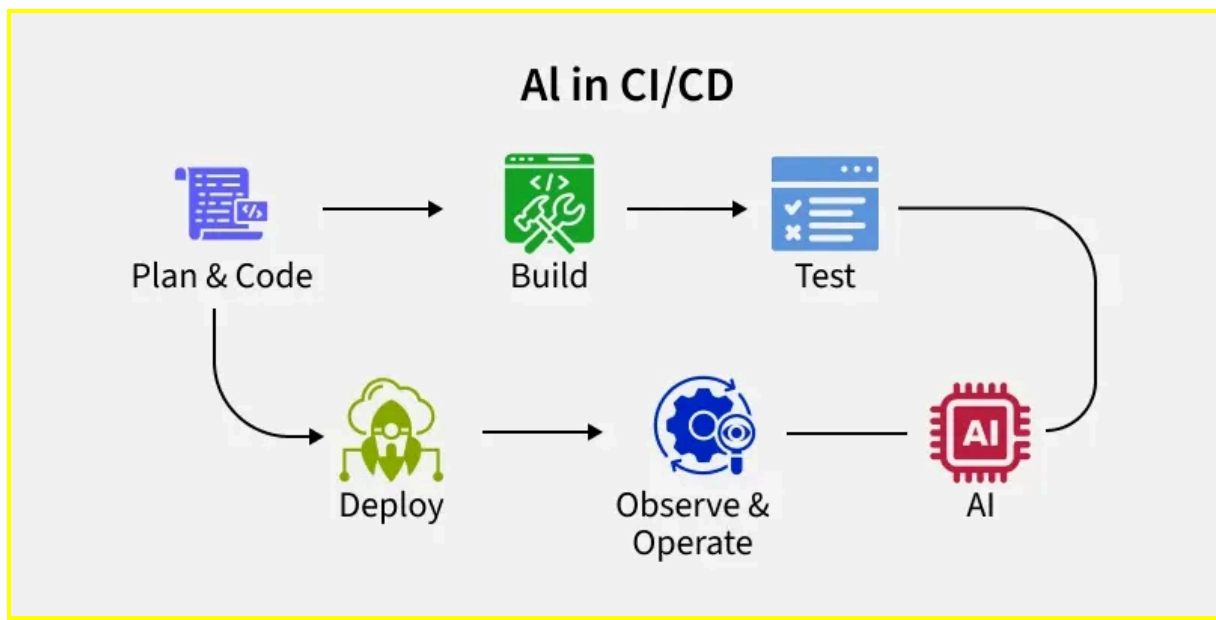
2. CI/CD Requirements

The system requires a CI/CD pipeline because it contains multiple components such as:

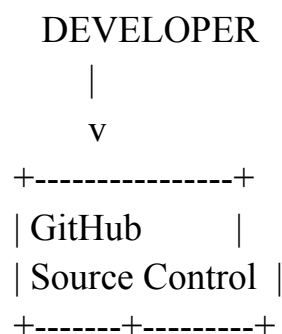
- IoT data collection services.
- AI/ML prediction services.
- Renewable asset monitoring APIs.
- Web dashboards.
- Database services.
- Cloud infrastructure.
- Alert and notification services.

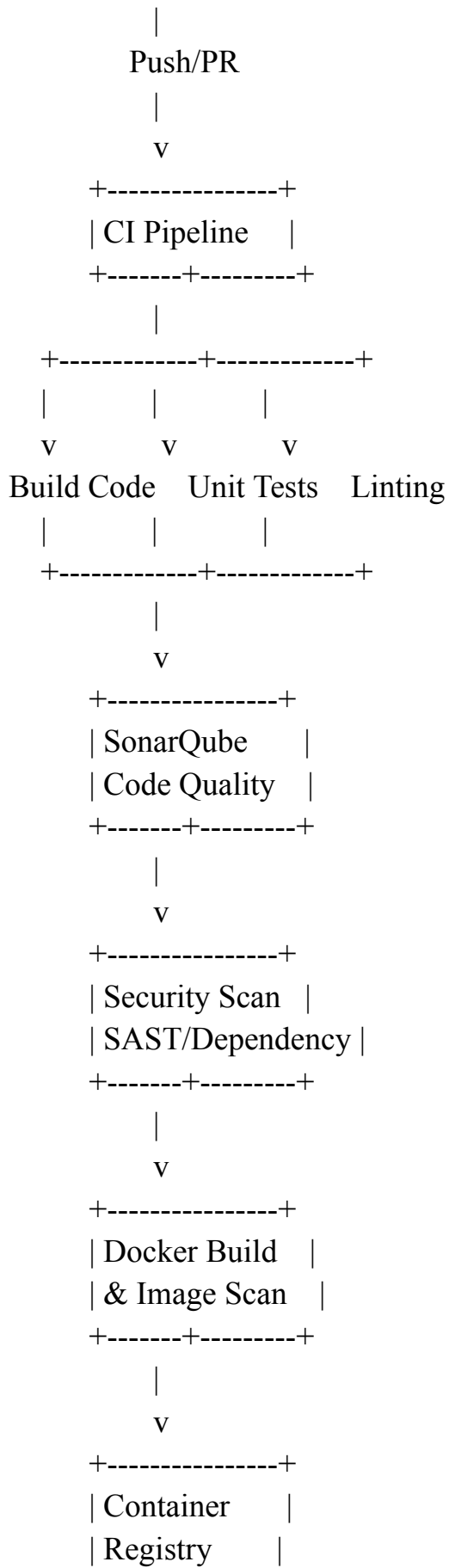
The CI/CD pipeline should:

1. Automatically detect code changes.
 2. Build the application after every commit.
 3. Perform automated unit and integration testing.
 4. Analyze source code quality.
 5. Scan dependencies and container images for vulnerabilities.
 6. Package applications into Docker containers.
 7. Deploy automatically to development and staging environments.
 8. Deploy approved versions to production.
 9. Monitor deployment health.
 10. Provide rollback when deployment failures occur.
-



3. Proposed CI/CD Pipeline Architecture





+-----+-----+

|

v

+-----+

| Deploy to |

| Development |

+-----+

|

v

+-----+

| Integration & |

| Performance Test|

+-----+

|

v

+-----+

| Staging |

| Environment |

+-----+

|

Approval/Gate

|

v

+-----+

| Production |

| Deployment |

+-----+

|

v

+-----+

| Monitoring & Alerts |

| Logs / Metrics |

+-----+

|

Failure detected

|

v

4. CI/CD Pipeline Stages

Stage 1: Source Code Management

Tool: GitHub + Git

Developers maintain the source code in a Git repository.

The repository may contain:

renewable-energy-monitoring/

- |
- |— frontend/
- |— backend/
- |— ai-model/
- |— iot-service/
- |— database/
- |— tests/
- |— docker/
- |— kubernetes/
- |— .github/
 - |— workflows/
- |— requirements.txt
- |— Dockerfile
- |— README.md

Developers create feature branches and submit pull requests before merging changes into the main branch.

Stage 2: Build

Tools: GitHub Actions, Python/Node.js, Maven or Gradle where applicable.

The pipeline automatically installs dependencies and builds the application.

Example activities:

- Install required packages.
- Compile application code.
- Validate configuration files.
- Build frontend and backend services.
- Prepare AI/ML dependencies.
- Generate build artifacts.

A failed build stops the pipeline and notifies the development team.

Stage 3: Automated Testing

Automated tests verify that the system works correctly after every code change.

Testing includes:

Unit Testing

- Test individual functions.
- Test API functions.
- Test AI prediction functions.
- Test sensor-data processing.

Integration Testing

- Test API and database communication.
- Test IoT data ingestion.
- Test AI service integration.
- Test dashboard-to-backend communication.

System Testing

- Verify complete monitoring workflows.
- Verify asset monitoring.
- Verify failure prediction.
- Verify maintenance alerts.

Regression Testing

- Ensure new changes do not break existing functionality.
-

Stage 4: Code Quality Analysis

Tool: SonarQube

SonarQube analyzes the source code for:

- Bugs.
- Code smells.
- Duplicated code.
- Maintainability issues.
- Reliability problems.
- Security vulnerabilities.

A quality gate is configured so that code containing serious issues cannot proceed to deployment.

Stage 5: Security Testing

Security checks are integrated into the pipeline.

Recommended tools include:

- **OWASP Dependency-Check** for vulnerable dependencies.
- **Trivy** for container image vulnerability scanning.
- **GitHub security features** for repository and dependency security.
- SAST tools for detecting insecure coding practices.

Security scans should be performed before deployment.

Stage 6: Docker Image Creation

Each major application component can be packaged as a Docker container.

Example:

IoT Service → Docker Image
Backend API → Docker Image
AI Prediction → Docker Image
Frontend → Docker Image

Docker provides consistent execution environments across development, testing, and production.

The images are tagged using a version number or Git commit identifier.

Example:

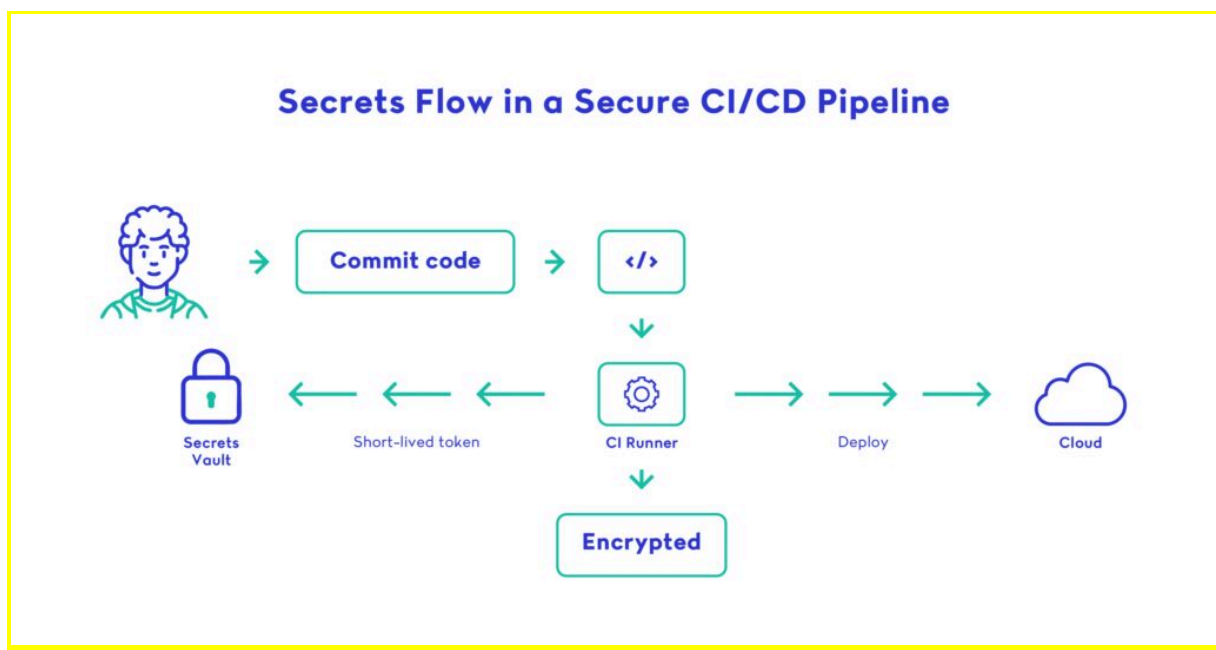
renewable-monitoring-api:v1.2.0
renewable-monitoring-ai:v1.2.0

Stage 7: Container Registry

Tool: GitHub Container Registry or another cloud container registry.

Successfully built and scanned images are stored in the container registry.

Only approved images should be available for production deployment.



5. Deployment Environments

Development Environment

The development environment is used by developers for initial validation.

Characteristics:

- Automatic deployment after successful CI.
- Test databases.
- Simulated IoT sensor data.
- Development configuration.
- Frequent deployments.

Testing/Staging Environment

The staging environment closely resembles production.

It is used for:

- Integration testing.
- System testing.
- Performance testing.
- Security validation.
- User acceptance testing.

Production Environment

The production environment handles real renewable energy monitoring.

It contains:

- Live IoT data.
- AI prediction services.
- Production database.
- Monitoring dashboard.
- Alert services.
- Cloud infrastructure.

Production deployment should require an approval gate unless a fully automated deployment strategy has been explicitly authorized.

6. Deployment Strategy

A **Blue-Green or Rolling Deployment** strategy can be used.

Rolling Deployment

New application instances are gradually deployed while existing instances continue serving users.

Old Version

|
v

Instance 1 —> New Version

Instance 2 —> New Version

Instance 3 —> New Version

|
v

All Instances Updated

This minimizes downtime and is suitable for a monitoring system that should remain continuously available.

Blue-Green Deployment

Two production environments are maintained:

Blue → Current Production

Green → New Version

After testing the Green environment, traffic is redirected from Blue to Green.

If a problem occurs, traffic can immediately be redirected back to Blue.

7. Automated Testing Strategy

Testing Type	Purpose	Pipeline Stage
Unit Testing	Validate individual functions	CI
Integration Testing	Verify component interaction	CI/Staging
API Testing	Validate backend APIs	CI/Staging
AI Model Testing	Validate prediction accuracy	CI/Staging
Database Testing	Validate database operations	CI
Security Testing	Identify vulnerabilities	CI
Performance Testing	Measure response and processing performance	Staging
Regression Testing	Verify existing functionality	Staging
Smoke Testing	Confirm deployment is working	Deployment
End-to-End Testing	Validate complete workflows	Staging

For AI functionality, model-specific validation should also check prediction accuracy, model version compatibility, input-data validation, and abnormal sensor-data handling.

8. Security Measures

Security must be integrated throughout the CI/CD pipeline.

Source Code Security

- Use protected branches.

- Require pull-request reviews.
- Avoid storing passwords in source code.
- Use secret management facilities.

Dependency Security

Automatically scan third-party libraries for known vulnerabilities.

Container Security

Scan Docker images before they are pushed to production.

Access Control

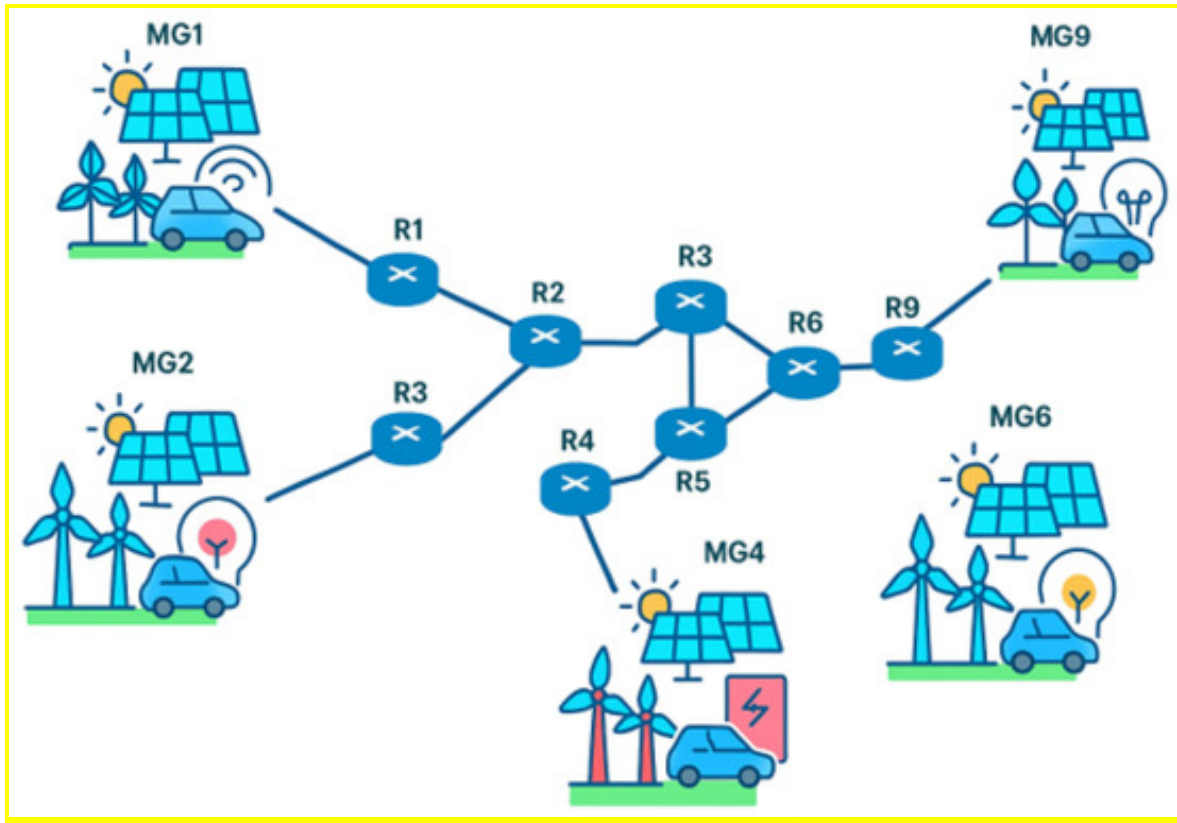
Use role-based access control for developers, testers, and deployment administrators.

Secrets Management

Sensitive information such as:

- Database passwords.
- API keys.
- Cloud credentials.
- Authentication tokens.

should be stored in secure CI/CD secrets or a dedicated secrets manager rather than inside source code.



9. Quality Gates

The pipeline should stop deployment when important quality conditions fail.

Example quality gates:

Build Successful



Unit Tests Passed



Code Quality Passed



Security Scan Passed



Container Scan Passed



Integration Tests Passed



Performance Acceptable



Production Approval



Deployment

This prevents defective or vulnerable software from reaching production.

10. Notification Mechanism

The CI/CD system should notify developers and administrators about pipeline results.

Notifications can be sent through:

- Email.
- Microsoft Teams.
- Slack.
- GitHub notifications.

Notifications should include:

- Build status.
 - Test status.
 - Security scan result.
 - Deployment status.
 - Deployment version.
 - Rollback status when applicable.
-

11. Rollback Mechanism

A rollback mechanism is essential because the system continuously monitors critical renewable energy assets.

Rollback can be triggered when:

- Application health checks fail.
- Deployment causes service errors.

- API response time becomes unacceptable.
- Critical tests fail.
- Container crashes occur.
- Database compatibility problems are detected.

Example:

Production Deployment

|
v

Health Check

|

+-----+-----+

|

Pass

|

Fail

|

v

|

v

Continue Rollback

|

v

Previous Version

The previous stable container image should be retained so that the system can quickly return to a known working version.

12. Example GitHub Actions Workflow

A simplified pipeline configuration can contain the following stages:

name: Renewable Energy CI/CD

on:

push:

branches: [main]

pull_request:

branches: [main]

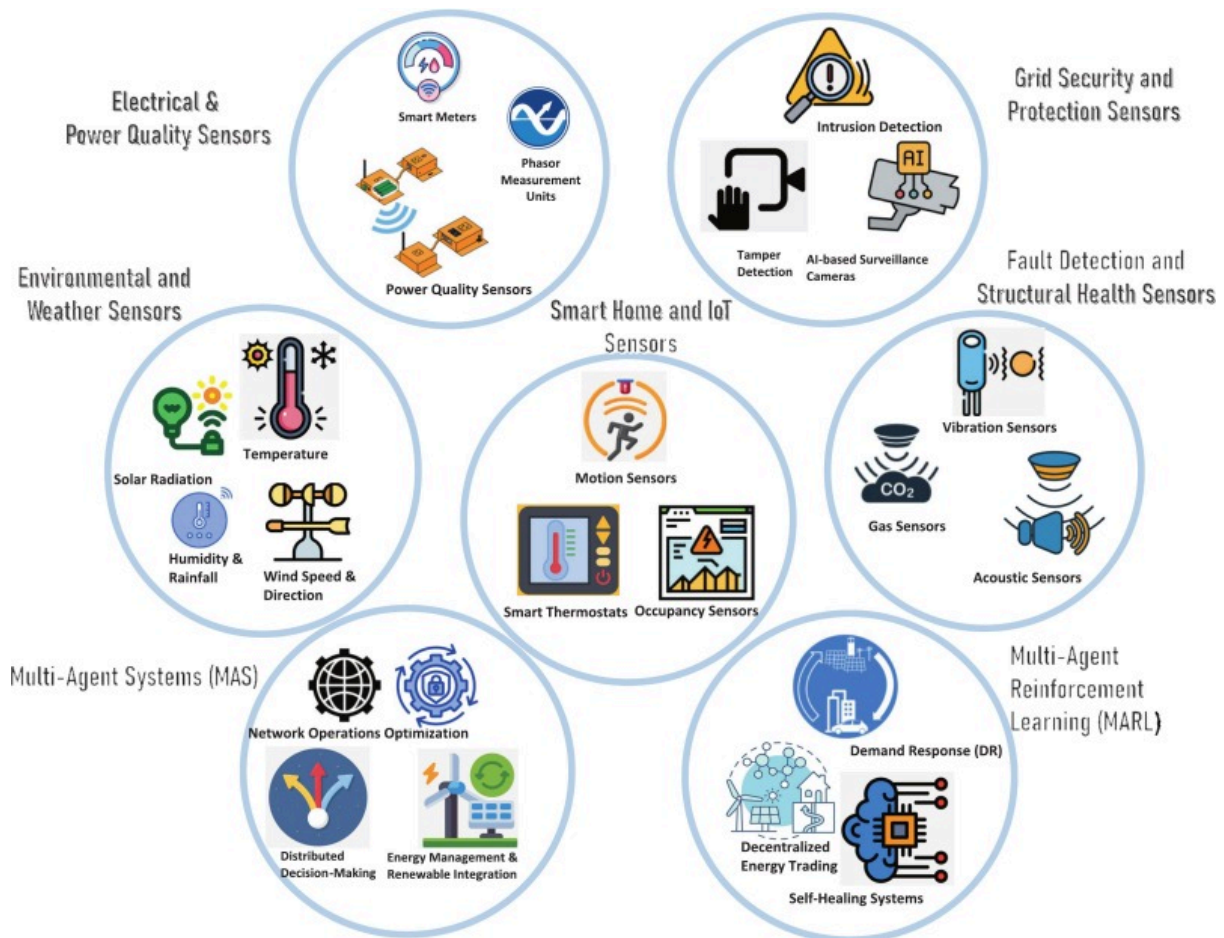
jobs:

build:

runs-on: ubuntu-latest

steps:

- name: Checkout Source
uses: actions/checkout@v4
- name: Setup Python
uses: actions/setup-python@v5
with:
python-version: "3.12"
- name: Install Dependencies
run: pip install -r requirements.txt
- name: Run Unit Tests
run: pytest
- name: Code Quality Check
run: |
pip install flake8
flake8 .
- name: Build Docker Image
run: docker build -t renewable-monitoring:\${{ github.sha }} .
- name: Security Scan
run: |
echo "Run container security scan"
- name: Deploy to Staging
run: |
echo "Deploy application to staging"
- name: Staging Tests



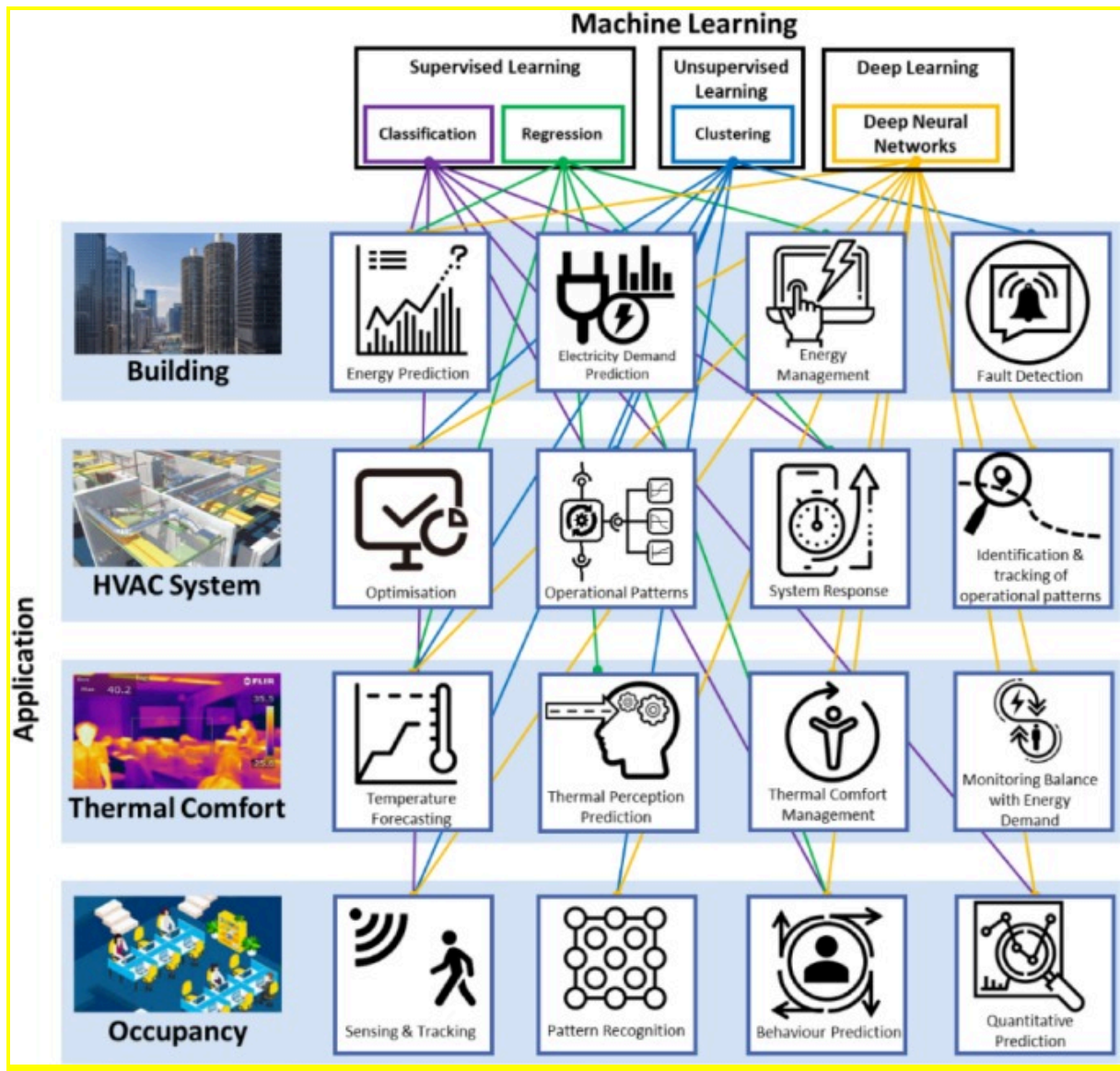
```
run: |
    echo "Run integration and smoke tests"

- name: Deploy Production
  if: github.ref == 'refs/heads/main'
  run: |
    echo "Deploy approved version to production"
```

This is a simplified example; production deployment should additionally use secure credentials, environment protection rules, image registries, health checks, and rollback automation.

13. Recommended Tools and Technologies

Pipeline Activity	Recommended Tool	Justification
Source Control	GitHub	Version control and collaboration
CI/CD	GitHub Actions	Automated workflow integration
Programming	Python / JavaScript	Suitable for AI and web services
Unit Testing	PyTest / Jest	Automated application testing
API Testing	Postman/Newman	API validation
Code Quality	SonarQube	Static code analysis
Security	OWASP / Trivy	Vulnerability detection
Containerization	Docker	Portable application deployment
Registry	GitHub Container Registry	Stores container images
Orchestration	Kubernetes	Scalable container deployment
Cloud	AWS/Azure/GCP	Infrastructure and scalability
Monitoring	Prometheus + Grafana	Metrics and visualization
Logging	ELK/OpenSearch	Centralized logging
Notifications	Teams/Slack/Email	Pipeline status alerts

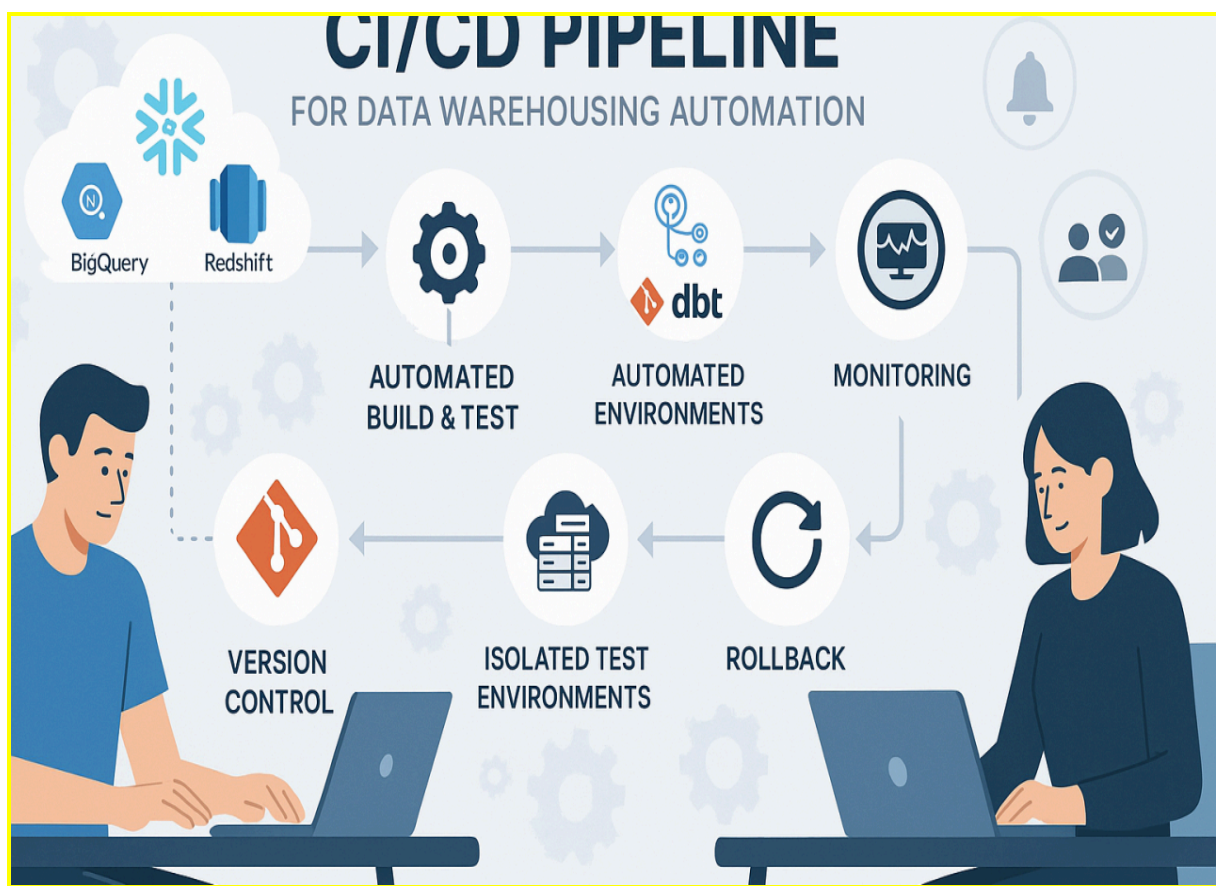


14. CI/CD Workflow

The complete workflow is:

1. Developer creates or modifies code.
2. Code is committed to a Git branch.
3. Pull request is created.
4. CI pipeline starts automatically.
5. Dependencies are installed.
6. Application is built.
7. Unit tests are executed.
8. Code quality is analyzed.
9. Security vulnerabilities are checked.

10. Docker images are created.
11. Container images are scanned.
12. Images are stored in the container registry.
13. Application is deployed to the development environment.
14. Integration and system tests are performed.
15. Application is deployed to staging.
16. Performance and acceptance tests are performed.
17. Production approval is obtained.
18. Application is deployed using rolling or blue-green deployment.
19. Health checks verify the deployment.
20. Monitoring systems track application performance.
21. If deployment fails, the previous stable version is restored.



15. Justification of the CI/CD Design

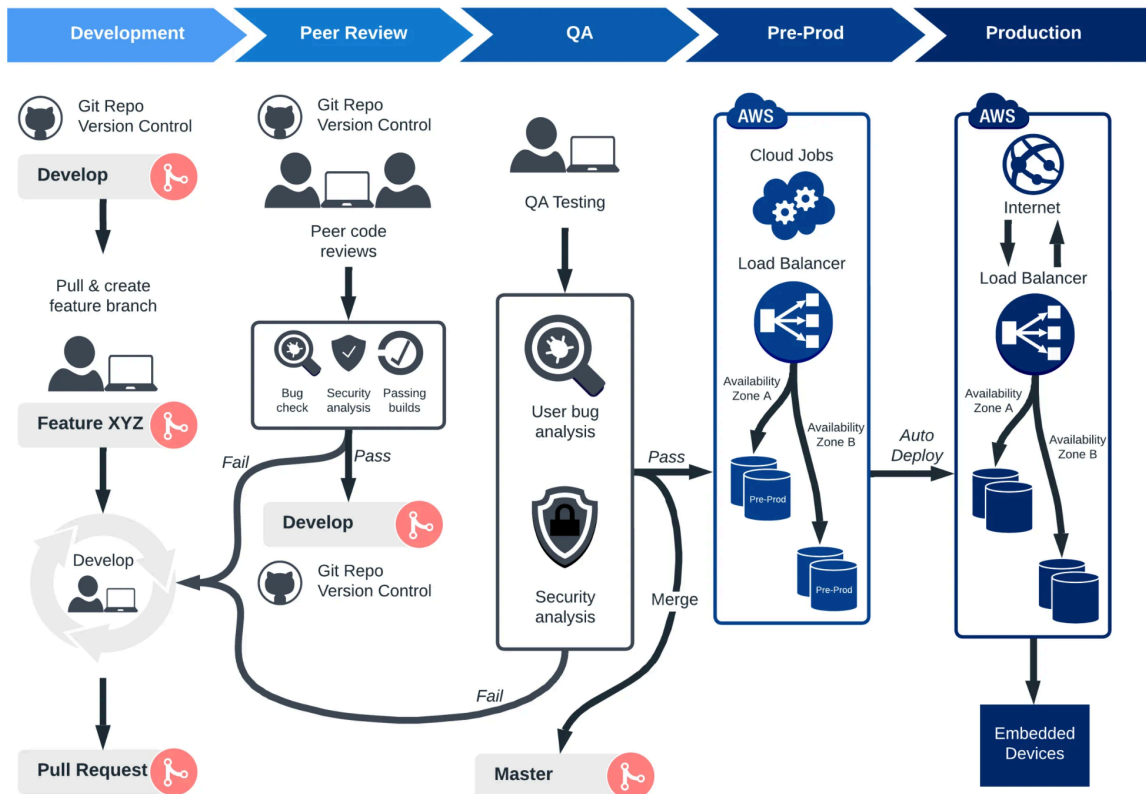
The proposed CI/CD architecture is appropriate for the renewable energy monitoring system because the platform requires frequent software updates while maintaining high availability.

GitHub and **GitHub Actions** provide integrated source control and automation. **Docker** ensures consistent application environments. **SonarQube** improves code quality, while **OWASP tools** and **Trivy** help identify security vulnerabilities. **Kubernetes** supports scalable deployment of monitoring and AI services. Automated testing reduces the possibility of defective software reaching production.

The use of staging environments and deployment approval provides additional protection before production release. Rolling or blue-green deployment reduces downtime, while automated health checks and rollback mechanisms improve system reliability.

16. Benefits of the Proposed CI/CD Pipeline

- Faster software releases.
 - Automated code integration.
 - Reduced manual deployment effort.
 - Early detection of defects.
 - Improved software quality.
 - Improved security.
 - Consistent deployment environments.
 - Reduced production downtime.
 - Faster rollback.
 - Better collaboration between development and operations teams.
 - Continuous delivery of AI and monitoring improvements.
 - Higher reliability of renewable energy monitoring services.
-



Conclusion

The proposed CI/CD pipeline provides an automated and reliable approach for developing, testing, securing, and deploying the **AI-Based Smart Renewable Energy Asset Performance Monitoring System**. The pipeline integrates source control, automated builds, unit and integration testing, code-quality analysis, security scanning, containerization, staging, production deployment, monitoring, and rollback.

By automating these activities, the system can deliver new monitoring features and AI improvements rapidly while maintaining reliability, security, scalability, and availability. The proposed architecture therefore supports the project's objective of maximizing renewable energy generation, reducing equipment downtime, improving predictive maintenance, and achieving efficient sustainable energy production.